PIIPS — Manual UAT Server Deployment Guide

Precision Intelligence Invoice Processing Suite — v2.1. FastAPI backend + React frontend, hosted under IIS as a reverse proxy to a Windows-service-hosted Uvicorn process.

1. Overview

The application runs as two pieces on one Windows server:

- **Backend** — a FastAPI app (`app.py`, exposed as `main:app`) run by Uvicorn, installed as a Windows Service via NSSM so it starts automatically and restarts on crash. It listens on `127.0.0.1:8000` (localhost only — not exposed directly).
- **IIS site** — a plain reverse proxy (URL Rewrite + Application Request Routing) that listens on the public port and forwards every request to the backend on port 8000. The built React frontend is also served from the same backend process, so IIS only ever talks to port 8000.

This guide reproduces that exact setup on a new UAT server, step by step, with no automated deployment scripts — every step is a manual action.

2. Prerequisites to install on the UAT server

Component	Purpose	Notes
Python 3.10.x (64-bit)	Runs the backend	Match the dev version (3.10.10) to avoid wheel-compatibility issues with paddlepaddle/opencv.
Node.js (LTS)	Builds the React frontend	Only needed if you build on the UAT server itself; skip if you copy a pre-built `frontend/dist` from a machine that already has Node.
ODBC Driver 17 for SQL Server	Backend→SQL Server connectivity (pyodbc)	Native driver, not installed via pip. Download from Microsoft and install with default options.
Poppler for Windows	PDF→image rasterisation (pdf2image)	Unzip anywhere (e.g. `C:\Poppler`) and add its `Library\bin` folder to the machine's **System** PATH environment variable (not just the user PATH).
IIS with URL Rewrite + Application Request Routing (ARR)	Reverse proxy to the backend	Installers are bundled in the repo: `required_softwares\rewrite_amd64_en-US.msi` and `required_softwares\requestRouter_amd64.msi`.
NSSM (Non-Sucking Service Manager)	Runs Uvicorn as a Windows Service	Bundled in the repo: `required_softwares\nssm-2.24\win64\nssm.exe`. No installation needed — it's a standalone exe.

> Install IIS itself first (Server Manager → Add Roles and Features → Web Server (IIS)) if it isn't already present, before installing URL Rewrite/ARR on top of it.

3. Copy the application to the UAT server

Copy the entire project folder to the UAT server, e.g. to `D:\PIIPS_App`. **Do not** copy these folders/files — they are machine-specific or regenerable:

- `venv\` — rebuild fresh on the UAT server (a venv is not portable between machines).
- `__pycache__\`, `logs\` — regenerated automatically.
- `config.json` — see §6, the database connection string inside it is encrypted for the *source* machine only and will not decrypt on the UAT server.

Do copy, if you want to carry over existing data:

- `format_model.json` and `model_backups\` — the trained invoice-format recognition model. Skip this if UAT should start with an untrained model and be trained fresh via the Training screen.
- `logo\` — static branding assets.

4. Set up the Python environment

Open a Command Prompt/PowerShell as Administrator, in the project folder on the UAT server:

```
cd /d D:\PIIPS_App
python -m venv venv
venv\Scripts\pip install --upgrade pip
venv\Scripts\pip install -r requirements.txt
```

> `requirements.txt` is tracked in the repo (a full `pip freeze` dump of the working dev environment, ~119 pinned packages) — use `pip install --no-deps -r requirements.txt` instead of a plain `-r requirements.txt` install: paddleocr declares `opencv-python<=4.6.0.66`, which otherwise conflicts with the pinned `opencv-python==4.8.1.78` and fails with `ResolutionImpossible`. `--no-deps` skips re-resolving each package's own dependency ranges since every dependency is already pinned explicitly.

Verify the interpreter works and can import the app before continuing:

```
venv\Scripts\python.exe -c "import app; print('OK')"
```

5. Build the frontend

From the `frontend` subfolder (either on the UAT server itself, if Node is installed there, or on any machine with Node and then copy the `dist` folder over):

```
cd frontend
```

```
npm install
npm run build
```

This produces `frontend\dist`, which the FastAPI backend serves directly — no separate web server or copy step is needed for the frontend itself.

6. Configure the database connection

> The connection string in `config.json` is encrypted at rest using Windows DPAPI at the local-machine scope. A `config.json` copied from another machine will have a connection string that **cannot be decrypted** on the UAT server — the app will treat the database as unconfigured.

On the UAT server, set the connection string fresh, either:

- **Via the app UI** (recommended) — start the backend (see §7–8 first), open the site, log in, go to **Database Configuration**, and paste in the .NET-style connection string for the UAT SQL Server instance. The app encrypts and saves it automatically.
- **Via config.json directly** — edit `config.json`'s `db_connection` field with a *plaintext* .NET-style connection string (e.g. `Server=UAT-SQL01;Database=PIIPS;User Id=piips_svc;Password=...;`). The app detects it isn't yet DPAPI-protected and encrypts it in place the next time it saves configuration.

Also set `folder_path` (the base folder for Input/Output/status folders, e.g. `D:\PIIPS_Data`) and, if Service First integration is needed on UAT, `sf_api_url` — both can also be set from the Folder Configuration / API Configuration screens after first login.

7. Install the backend as a Windows Service (NSSM)

Run these from an elevated Command Prompt (adjust the paths for the UAT install location):

```
cd /d D:\PIIPS_App\required_softwares\nssm-2.24\win64

nssm.exe install PIIPS_Backend "D:\PIIPS_App\venv\Scripts\python.exe" "-m uvicorn
main:app --host 127.0.0.1 --port 8000"
nssm.exe set PIIPS_Backend AppDirectory "D:\PIIPS_App"
nssm.exe set PIIPS_Backend Start SERVICE_AUTO_START
nssm.exe set PIIPS_Backend AppStdout "D:\PIIPS_App\logs\service_stdout.log"
nssm.exe set PIIPS_Backend AppStderr "D:\PIIPS_App\logs\service_stderr.log"
nssm.exe set PIIPS_Backend AppExit Default Restart

net start PIIPS_Backend
```

Confirm it's running and healthy:

```
Invoke-RestMethod http://127.0.0.1:8000/health
```

Expected response: `{"status":"Healthy"}`. The first start can take 15–30 seconds while PaddleOCR initialises — retry the health check a few times before assuming a failure.

8. PaddleOCR model cache (important — avoid on first run)

> NSSM services run as **Local System** by default, a different Windows account from whoever is logged in interactively. PaddleOCR downloads its recognition models to the *current user's* profile (`%UserProfile%\.paddleocr`) the first time it runs — under Local System that's `C:\Windows\System32\config\systemprofile\.paddleocr`, which is empty on a fresh server. If the UAT server has no internet access, the first OCR call will fail trying to download the models.

Two ways to avoid this:

1. **Pre-seed the cache** — on a machine that already has the models (e.g. the dev machine), copy `%UserProfile%\.paddleocr` to `C:\Windows\System32\config\systemprofile\.paddleocr` on the UAT server before first start.
2. **Let it download once** — if the UAT server does have outbound internet access, simply let the first request download the models (a one-time delay), then no further action is needed.

9. Configure the IIS reverse-proxy site

> **Shared UAT server note:** if the UAT server already hosts another site (e.g. an existing .NET MVC application) under IIS, PIIPS must be given its **own, currently-unused port** — it must not reuse that application's port or its Default Web Site binding. Both applications otherwise coexist without conflict: PIIPS's app pool runs **No Managed Code** (it only proxies to Uvicorn) and the NSSM-hosted backend listens on `127.0.0.1:8000`, a port private to PIIPS. If the existing .NET app (or anything else) also happens to be using port 8000 internally, change PIIPS's `--port` in §7 and the matching target port in `web.config`'s rewrite rule (§9.3) to a free one, consistently in both places.

9.1 Check what ports are already bound before picking one for PIIPS, so it doesn't collide with the existing application's site:

```
Import-Module WebAdministration
Get-Website | Select-Object Name, State,
@{n='Bindings';e={($_.bindings.Collection | ForEach-Object {
$_.bindingInformation }) -join '; '}}
netstat -ano | findstr LISTENING
```

Pick a port for PIIPS that appears in neither list (5000 is used only as a reference example throughout this guide).

9.2 Enable ARR proxying (one-time, server-wide — if another site on this server already proxies via ARR, this is likely already enabled and this step is a no-op): in IIS Manager, click the server node (top level) → **Application Request Routing Cache** → **Server Proxy Settings...** (right pane) → check **Enable proxy** → Apply.

9.3 Create the site, using the free port found in §9.1:

```
New-Website -Name "PIIPS_AI" `
    -PhysicalPath "D:\PIIPS_App" `
    -Port 5000 `
```

```
    -ApplicationPool "PIIPS_AI" -Force
```

The application pool can be left at its defaults — **.NET CLR version: No Managed Code**, **Managed pipeline mode: Integrated**, identity: **ApplicationPoolIdentity**. No managed code runs in this pool; it only proxies. This is a separate app pool from the existing .NET MVC application's, so neither one's runtime/recycle settings affect the other.

9.4 Reverse-proxy rule: the physical path already contains a `web.config` with the rewrite rule below. If deploying to a folder without it, create `web.config` in the site's physical path with exactly this content:

```
<?xml version='1.0' encoding='utf-8'?>
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <rule name="ReverseProxy" stopProcessing="true">
          <match url="(.*)" />
          <action type="Rewrite" url="http://127.0.0.1:8000/{R:1}" />
        </rule>
      </rules>
    </rewrite>
  </system.webServer>
</configuration>
```

> The target `http://127.0.0.1:8000` must match the `--port` the NSSM service was set up with in §7.

10. Verification / smoke test

1. Confirm the service is running: `Get-Service PIIPS_Backend` should show **Running**.
2. Confirm the backend answers directly: `Invoke-RestMethod http://127.0.0.1:8000/health` → `{"status":"Healthy"}`.
3. Confirm the IIS site is started: `Get-Website -Name "PIIPS_AI"` should show **Started**.
4. Confirm the proxy end-to-end: `Invoke-WebRequest http://localhost:5000/health -UseBasicParsing` → HTTP 200, same JSON body.
5. Open `http://<uat-server>:5000` in a browser, log in, and confirm the Dashboard loads.
6. Go to **Folder Configuration** and confirm/set the base folder path — the app creates the `Input`, `New_Format`, `Trained_format`, `Output` and per-status subfolders automatically the first time it's saved.
7. Go to **Database Configuration** and confirm the connection test succeeds (see §6 if it was skipped earlier).
8. Process one sample PDF end-to-end (Dashboard → Start) and confirm it completes without errors.

11. Common problems

Symptom	Cause	Fix
IIS shows **502.3 Bad Gateway** right after a service restart	Uvicorn/ PaddleOCR is still initialising (10–30s)	Wait and retry; not a real failure if it clears within a minute.
IIS shows **502.3 Bad Gateway** persistently	ARR proxy not enabled, or backend not listening on the port web.config targets	Re-check §9.2 (Enable proxy) and that `Invoke-RestMethod http://127.0.0.1:8000/health` works directly.
Database Configuration always shows "not configured" after copying config.json	DPAPI-encrypted connection string from a different machine	Re-enter the connection string on this machine (§6).
First PDF processed fails with a network/DNS error mentioning paddleocr.bj.bcebos.com	Local System's PaddleOCR cache is empty and the server has no internet access	Pre-seed the cache as described in §8.
`pyodbc.InterfaceError` / "Data source name not found"	ODBC Driver 17 for SQL Server not installed	Install it from Microsoft (see §2).
PDF processing fails with a poppler/pdfinfo error	Poppler not on the System PATH, or only on the interactive user's PATH	Add Poppler's `Library\bin` to the **System** PATH (not User PATH) and restart the PIIPS_Backend service so it picks up the new PATH.
`[IM002] Data source name not found and no default driver specified`	ODBC Driver 17 for SQL Server not installed (the app hardcodes this exact driver name — installing v18 will not fix it, the name string won't match)	`Get-OdbcDriver -Name "ODBC Driver 17 for SQL Server"` to check; if missing, install from `https://go.microsoft.com/fwlink/?linkid=2200732` with `msiexec /i msodbcsql17.msi /passive /norestart IACCEPTMSODBCSQLLICENSETERMS=YES`. On a server with no internet access, download the MSI on a machine that has access and copy it over via admin share or RDP drag-and-drop first.

Site loads fine at `http://localhost:<port>` on the server itself, but times out from every other machine	No inbound firewall rule for the site's port	`New-NetFirewallRule -DisplayName "PIIPS IIS <port>" -Direction Inbound -Protocol TCP -LocalPort <port> -Action Allow`, then confirm from a *different* machine with `Test-NetConnection <server-ip> -Port <port>`.
A UI change (new menu item, redesigned screen) doesn't appear after copying updated files and restarting the service	PIIPS is a PWA with an active service worker (`frontend/public/sw.js`); a normal Ctrl+F5 does **not** reliably bypass an already-installed service worker in a regular browser tab	Open DevTools (F12) → Application tab → Service Workers → Unregister (or Application → Storage → Clear site data), then reload. Confirm first by opening the site in an Incognito/Private window, where no service worker is registered yet.

12. Source control workflow (GitHub)

The project is tracked in a private GitHub repository with three long-lived branches:

Branch	Purpose
`Development`	The working branch — every local code change lands here first.
`uat`	What the UAT server actually runs. Promoted from `Development` (today: manually, or via the Publish menu's UAT flow — see §13).
`live`	What production runs. Promoted from `uat` only — never directly from `Development`. As of this writing `live` has no application code merged into it yet (placeholder only); the Publish menu's Live option currently deploys from `uat` until that changes.

> Sample invoice PDFs (`sample_input/`) and the generated manual PDFs are intentionally gitignored — they live on disk but are never pushed to GitHub. `config.json` and `config_nonencrypted.json` are gitignored too, since they can contain real database credentials.

13. The Publish menu (Super Admin)

Super Admin users have a **Publish** item in the sidebar (Admin group) that automates the build/stage step for a new release, without touching any remote server directly:

1. **Root path (local)** — a folder on *this* machine where a build gets staged, e.g. `D:\WorkSpace\Projects\Python\PIIPS_Published`. Each environment gets its own subfolder (`\uat`, `\live`). Set it once via **Save root path**.

2. **Environment** — choose **UAT** or **Live**, then click **Publish to <env>**.

What happens on click, per environment:

- **UAT** — commits and pushes any uncommitted local changes onto `Development`, pulls `Development`, runs `npm install` + `npm run build` for the frontend, then copies everything (minus `venv`, `node_modules`, `.git`, logs, sample data, etc.) into the staged `\uat` folder. It does **not** touch the `uat` git branch.
- **Live** — a pure promotion: checks out and pulls the `uat` git branch as-is (no Development involvement), builds, and copies into the staged `\live` folder.

> Publish only *stages* a build locally — it does not copy anything onto the real UAT/Live server or restart any service there. That is still a manual step: mirror the staged folder onto the server and restart its service.

On the destination server (RDP in), sync the staged build with a **mirror** copy — not a plain overwrite, which can leave stale files behind (an old content-hashed JS bundle, or a skipped `index.html`) and silently serve outdated content:

```
$StagedBuild = "D:\WorkSpace\Projects\Python\PIIPS_Published\uat"
$Destination = "E:\PIIPS_Application"   # the real server path

robocopy $StagedBuild $Destination /MIR /XJ /R:2 /W:5 /MT:8 `
    /XF "config.json" "config_nonencrypted.json" "*.log"

Restart-Service PIIPS_Backend
Start-Sleep -Seconds 15
Invoke-RestMethod http://127.0.0.1:8000/health
```

`/MIR` copies what's new/changed *and* deletes anything in the destination no longer present in the source, in one step. `config.json` is excluded from both the copy and the delete pass, so the server keeps its own DPAPI-encrypted connection string. See §11's last row if the UI still looks stale after this — that's almost always the service worker cache, not a bad copy.

> Full details and troubleshooting for both the initial project copy (§3) and this day-to-day sync live in `Deploy_To_Network.txt` at the project root.

14. Setting up a second database (e.g. PIIPS_UAT)

UAT can run against its own database rather than sharing the dev database. To create one with master/lookup data only (no transaction history):

1. Create the empty database on the target SQL Server instance.
2. Bootstrap its schema by pointing `database.ensure_menu_schema(force=True)` at it (temporarily swap the connection string, run once, swap back) — this creates every table/type/stored-procedure the app needs from a completely empty database.
3. Copy master data across with cross-database `INSERT ... SELECT` (same SQL Server instance, 3-part naming) for: `tbl_status`, `tbl_InvoiceType`, `tbl_UserType`, `tbl_SheetColumn`, `tbl_FieldMapping`, `tbl_Template`/`tbl_TemplateStaticValue` (filter by

`InvoiceType` if only one type should carry over), and `tbl_User` (filter to the specific accounts UAT needs). Use `SET IDENTITY_INSERT <table> ON` around each insert to preserve exact IDENTITY values, since foreign keys (e.g. `tbl_User.UserTypeID`) reference them directly.

> Two columns on `tbl_Purchase_Header` (`Last_Updated_No`, `PO_Number_Format`) are added lazily — the first time an invoice is actually *saved* through the app, not just when the schema is bootstrapped. On a database that already has header rows from before this self-heal logic existed (or copied in from elsewhere), add both columns directly and backfill them by hand if the Excel download's **No.** field is coming back without its PO Number Format prefix:

```
ALTER TABLE dbo.tbl_Purchase_Header ADD PO_Number_Format NVARCHAR(100) NULL;
ALTER TABLE dbo.tbl_Purchase_Header ADD Last_Updated_No NVARCHAR(200) NULL;
-- then backfill each blank [No.] row with its template's PO_Number_Format
-- + a zero-padded sequence number, e.g. PO-2627-000001, PO-2627-000002, ...
```

Appendix — quick command reference

```
REM Check/restart the backend service
Get-Service PIIPS_Backend
Restart-Service PIIPS_Backend

REM Check/restart the IIS site
Get-Website -Name "PIIPS_AI"
Restart-WebItem "IIS:\Sites\PIIPS_AI"

REM Health checks
Invoke-RestMethod http://127.0.0.1:8000/health
Invoke-WebRequest http://localhost:5000/health -UseBasicParsing

REM Service logs
Get-Content D:\PIIPS_App\logs\service_stdout.log -Tail 50
Get-Content D:\PIIPS_App\logs\service_stderr.log -Tail 50
```